

TECHNICAL TEST

Document Tree

Tree-based Document Management with Django & DRF

Context

You are joining the backend team of a B2B SaaS platform in the French pharmaceutical industry. The platform connects three types of users:

- **Laboratories:** Pharmaceutical brands (e.g., Nuxe, Bioderma).
- **Groups:** Networks or chains of pharmacies.
- **Pharmacies:** Individual retail pharmacies that belong to a Group.

Each entity creates and manages many different types of documents: generic files, promotional flyers, commercial terms, etc. Currently, these items are scattered and difficult to find.

The Objective: Build a cross-entity tree-based document management system. Every entity has its own tree structure but can share specific nodes (folders or files) with other entities.

A **Pharmacy** should see its own documents in its tree, as well as everything shared with it by its **Group** or by a **Laboratory**.

- **Example:** A Group creates a folder named "2025 Terms" containing commercial conditions and shares it with all its member Pharmacies. Each Pharmacy sees this folder in its own tree in **read-only** mode.

Example of a tree structure viewed by a Pharmacy belonging to the "CPC" Group:

- My documents
 - VAT Certificate (*Owned by the Pharmacy — Document Leaf #5*)
 - CPC (*Shared by Group*)
 - 2025 Terms (*Shared in read-only*)
 - General Terms (*CommercialCondition Leaf #12*)
 - Group Flyers
 - Sun Care Flyer (*Flyer Leaf #38*)
 - Nuxe (*Shared by Laboratory*)
 - Spring Operation
 - Product Brief (*Shared in read-only — Document Leaf #2391*)
-

Existing Business Objects

These models already exist in the project. **You do not need to modify them:**

```
from django.db import models

class Groupement(models.Model):
    name = models.CharField(max_length=255)

class Pharmacy(models.Model):
    name = models.CharField(max_length=255)
    groupement = models.ForeignKey(
        'Groupement',
        null=True,
        blank=True,
        on_delete=models.SET_NULL
    )

class Laboratory(models.Model):
    name = models.CharField(max_length=255)
    code = models.CharField(max_length=50, unique=True)

# --- Document Objects ---

class Document(models.Model):
    laboratory = models.ForeignKey('Laboratory', on_delete=models.CASCADE)
    name = models.CharField(max_length=255)
    file = models.FileField(upload_to='documents/')
    created_at = models.DateTimeField(auto_now_add=True)

class Flyer(models.Model):
    laboratory = models.ForeignKey('Laboratory', on_delete=models.CASCADE)
    title = models.CharField(max_length=255)
    image = models.ImageField(upload_to='flyers/')
    start_at = models.DateField()
    end_at = models.DateField()

class CommercialCondition(models.Model):
    laboratory = models.ForeignKey('Laboratory', on_delete=models.CASCADE)
    name = models.CharField(max_length=255)
    text = models.TextField()
    year = models.PositiveSmallIntegerField()
```

Phase 1: Architecture Document (1 to 2 hours)

Before writing any code, write a document that answers the following questions:

Question 1 — Tree Structure

Which storage strategy do you choose for the tree structure? (Adjacency List, MPTT, treebeard, Closure Table, Nested Sets...) Justify your choice by explaining the read/write trade-offs and the specific use cases that guided your decision.

Question 2 — Polymorphic Owner

A node can belong to a Laboratory, a Group, or a Pharmacy. How do you model this owner without using hardcoded Foreign Keys for each type? Is this the same mechanism used for the leaf content, or a different one? Why?

Question 3 — Sharing Model

How do you model node sharing? A Group can share with "all its member Pharmacies" or with "specific Pharmacies". A Laboratory can share with an entire Group or with individual Pharmacies. Propose a model and explain its limitations.

Question 4 — Content Polymorphism

How do you model a leaf that can point to a Document, a Flyer, or a CommercialCondition—without hardcoded Foreign Keys for each type? Present your chosen model and explain why you selected it.

Question 5 — Extensibility

In 6 months, we will add a new type of object: **Contract**. How does your architecture handle this change? Is a database migration necessary? Which parts of the code will be affected?

Question 6 — Permissions and Integrity

How do you guarantee that a Pharmacy cannot modify a node shared by its Group? And how do you prevent a node from being shared with a Pharmacy that does not belong to the correct Group?

Question 7 — Aggregated View

When a Pharmacy loads its tree, it must see its own nodes AND the nodes shared with it (by its Group or by Laboratories). How do you build this aggregated view efficiently? What are the performance implications?

Question 8 — API Design

How do you represent the tree structure in a REST API? (Flat list + parent ID, nested JSON, filesystem path...) Provide the signatures for the 3–4 main endpoints. How does the API distinguish between owned nodes and shared nodes?

Phase 2: Proof of Concept (3 to 4 hours)

Implement your architecture.

- **Django Models** (`document_tree/models.py`) containing the ownership and sharing relationships.
- **At least 3 functional DRF endpoints:**

- List the children of a node (tree navigation).
 - Resolve a leaf (return the data of the underlying object).
 - Aggregated view for a given entity (its own nodes + nodes shared with it).
 - **Tests** covering these endpoints (the nominal/success case + at least one error case for each).
 - Sharing endpoint: share a node with one or more entities.
 - Breadcrumb endpoint (path from root -> current node).
 - Moving a node inside the tree.
 - "Share with all pharmacies in the group" option (without explicitly listing them).
-

Expected Deliverables

1. **Architecture Document** (Markdown, Google Doc, or PDF).
2. **Developed Prototype.**
3. If you use AI, include **your conversations with the tool.**